653. Two Sum IV Input is a BST

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	EASY	Solved: August 3, 202

Problem Summary

Given the root of a BST and an integer k, return true if there exist two nodes whose values sum to k.

Approach

DFS traversal with a HashSet. For each node, check if its complement (k - root.val) is already in the set. If yes → pair found. If no → add current value to set and continue.

Algo

1. Initialize HashSet<Integer> set.
2. If root == null return false.
3. If set.contains(k - root.val) return true.
4. set.add(root.val).
5. Return findTarget(root.left, k) || findTarget(root.right, k).

Key Insights

- Same HashSet complement trick as **Two Sum (LC 1)** — k - root.val is the complement.
- Add to set AFTER checking — prevents using the same node twice (e.g., k=10, node value 5 → 5+5=10 should not count).
- Any traversal order works — pre/in/post/BFS all valid since you just need to visit every node.
- || short circuits — stops as soon as pair is found.

Time Complexity & Space Complexity

O(n) — every node visited once.

O(n) — HashSet stores all node values in worst case.

Approaches

- **Brute Force O(n²)**: For every pair of nodes check if they sum to k. O(n²) time.
- **Better O(n) space — Array**: In-order traversal → sorted array → two pointer approach. O(n) time O(n) space.
- **Optimal O(n) space — HashSet**: DFS with HashSet — for each node check if complement k - root.val already exists. O(n) time O(n) space.